



## Individual assignment cover sheet

|                       |                                                |
|-----------------------|------------------------------------------------|
| Course Name:          | Master of Information Systems (Cyber Security) |
| Unit Number and Name: | MIS5303 Securing Software                      |
| Lecturer Name:        | Dr Omid Haass                                  |
| Assignment No.:       | Assessment 1 - Practical Labs (Lab 5)          |

### Declaration:

By submitting this declaration, I, the below-named student, declare the following:

- ✓ Except where appropriately acknowledged, this assignment is my own work and has been expressed in my own words.
- ✓ Where I have made use of generative AI software, I have done so appropriately and cited it accurately as per the '**HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students**' document found in the LMS, See: [HE-AP19 Generative Artificial Intelligence \(AI\) Guidelines for Students.pdf \(apexhe.nsw.edu.au\)](https://apexhe.nsw.edu.au/HE-AP19_Generative_Artificial_Intelligence_(AI)_Guidelines_for_Students.pdf).
- ✓ This assignment in its entirety, or this particular part of it, has not previously been submitted by me or any other person for assessment on this or any other course of study.
- ✓ I have read the Student Academic Misconduct Policy and Procedure found at <https://apexhe.nsw.edu.au/policiesandprocedures/>, and I understand the significance of academic integrity and the consequences of breaching it.

### Use of Generative Artificial Intelligence (AI)

Please tick one of the following options:

- ✓ **Declaration 1:** I acknowledge using AI tool(s) to generate content that are included in this work in modified forms.
- ✓ **Declaration 2:** I acknowledge the use of AI tool(s) in the preparation of material that aids the process of completion of this assignment (e.g. for self-study or background reading). No materials are submitted as original work on our part.
- ✓ **Declaration 3:** I did not use any content generated by AI tools in the preparation of this assignment

If your response was **Declaration 1** or **Declaration 2**, please complete the following:

|                                                                                                                                                                  | Student's Response |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <b>AI Tool(s) Used</b> (e.g., ChatGPT, Grammarly, DALL·E 2, Pi Assistant, Lumen5, Looka, etc.)                                                                   |                    |
| <b>Purpose of Use</b> (Explain specifically what the tool was used for – e.g., idea generation, grammar correction, image generation, summarizing content, etc.) |                    |

|                                                                                                                                                                       |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>List all prompts you have used</b>                                                                                                                                 |  |
| <b>Extent of Modification</b> (Explain if and how you rewrote or modified the AI-generated content. E.g., rephrased, fact-checked, rearranged, quoted suitably, etc.) |  |
| <b>Consider the manners in which your work is different from AI-generated contents.</b>                                                                               |  |
| <b>How have you modified the generative AI's output?</b>                                                                                                              |  |

Note: All AI usage must be acknowledged and comply with the HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students. Unacknowledged use of AI tools constitutes academic misconduct.

|                           |                      |
|---------------------------|----------------------|
| <b>Student ID Number:</b> | <b>Student Name:</b> |
| 240135                    | Ashraful Islam       |

**Signed:**

**Date: 15-Aug-26**



Recoverable Signature

X Ashraful

Name

Signed by: 341b0587-a755-4806-a88b-56fe95828d8f

## MIS5303 Securing Software

### Lab 5 — Cryptography, Hashing & Data Protection

Assessment 1 — Supervised Practical Lab Submission

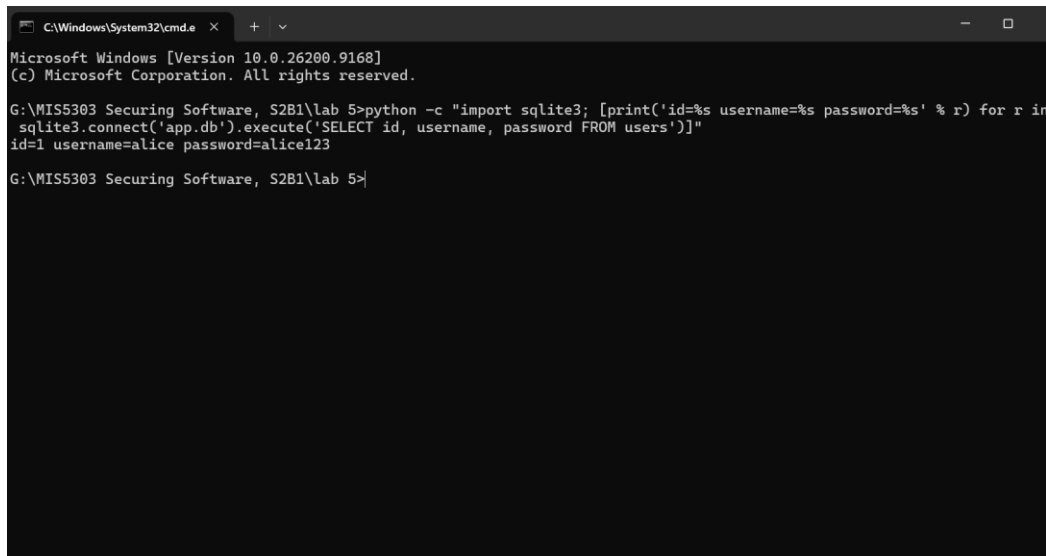
---

| Field              | Detail                    |
|--------------------|---------------------------|
| Name               | Ashraful Islam            |
| Student ID         | 240135                    |
| Lab number         | Lab 5 (Week 6)            |
| Date of completion | 15/08/2026                |
| Subject            | MIS5303 Securing Software |
| Lecturer           | Dr Omid Haass             |

## 1. Where Sensitive Data Is Handled (Baseline)

| ID | Data             | Where                | Baseline state                 | Risk                                                    |
|----|------------------|----------------------|--------------------------------|---------------------------------------------------------|
| D1 | User password    | register() / login() | Stored & compared in plaintext | DB leak exposes every password instantly                |
| D2 | Note content     | add_note() / index() | Stored in plaintext            | DB leak or backup theft exposes personal/clinical notes |
| D3 | Session identity | Flask session cookie | No HttpOnly/SameSite flags     | Cookie theft via XSS or cross-site misuse               |

### Evidence — before (baseline app.py)

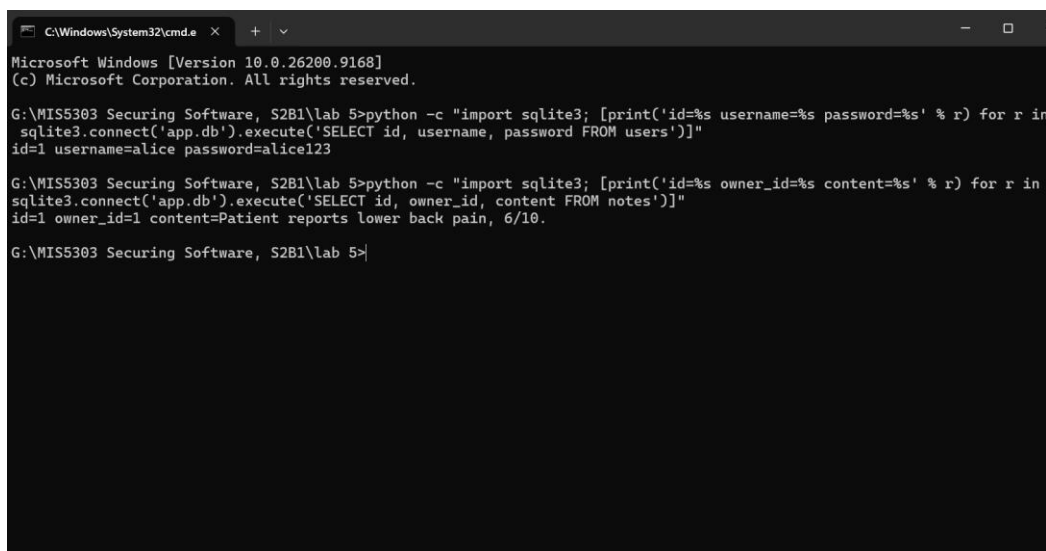


```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123

G:\MIS5303 Securing Software, S2B1\lab 5>|
```

Figure 1: alice's password stored in plaintext in app.db.



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s owner_id=%s content=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, owner_id, content FROM notes')]"
id=1 owner_id=1 content=Patient reports lower back pain, 6/10.

G:\MIS5303 Securing Software, S2B1\lab 5>|
```

Figure 2: alice's note content stored in plaintext in app.db.

## 2. Remediation — Secure Code (Before / After)

### 2.1 Password hashing (bcrypt)

Before:

```
conn.execute("INSERT INTO users (username,password) VALUES (?,?)", (username, password))
```

After:

```
password = request.form["password"].encode("utf-8")
hashed = bcrypt.hashpw(password, bcrypt.gensalt(rounds=12))
c.execute("INSERT INTO users (username,password) VALUES (?,?)", (username, hashed))
# login: bcrypt.checkpw(password, row["password"].encode())
```

*Why: bcrypt is a deliberately slow, adaptive hash with a built-in per-password salt, so identical passwords produce different hashes and offline brute-force is far harder than with a fast hash like MD5/SHA-256.*

### 2.2 Note encryption (Fernet)

Before:

```
c.execute("INSERT INTO notes (owner_id, content) VALUES (?, ?)", (session["user_id"], content))
```

After:

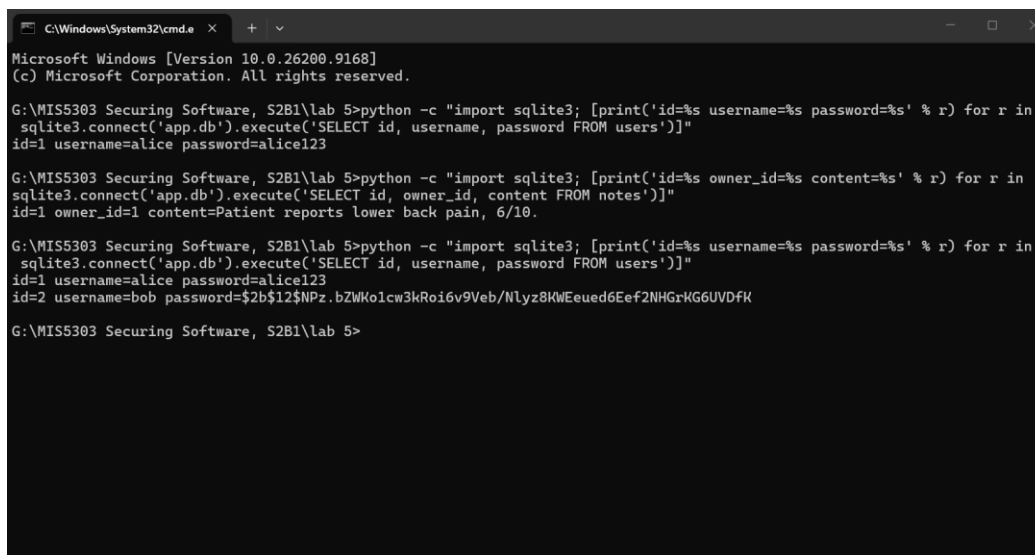
```
encrypted_content = FERNET.encrypt(content.encode("utf-8"))
c.execute("INSERT INTO notes (owner_id, content) VALUES (?, ?)", (session["user_id"], encrypted_content))
# display: FERNET.decrypt(row["content"].encode()).decode("utf-8")
```

*Why: Fernet provides authenticated symmetric encryption (AES-128 in CBC mode with an HMAC), so note content is unreadable at rest and any tampering with the ciphertext is detected on decryption.*

### 2.3 Session cookie hardening

```
app.config.update(SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE="Lax",
                  PERMANENT_SESSION_LIFETIME=timedelta(minutes=15))
```

## Evidence — after (protected app\_fixed.py)



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s owner_id=%s content=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, owner_id, content FROM notes')]"
id=1 owner_id=1 content=Patient reports lower back pain, 6/10.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123
id=2 username=bob password=$2b$12$NPz.bZWk0lcw3kRoi6v9Veb/Nlyz8kWEued6Eef2NHGrKG6UVDfK

G:\MIS5303 Securing Software, S2B1\lab 5>
```

Figure 3: alice's old plaintext password next to bob's new bcrypt hash (\$2b\$12\$...) in the same database.

```

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s owner_id=%s content=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, owner_id, content FROM notes')]"
id=1 owner_id=1 content=Patient reports lower back pain, 6/10.

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s username=%s password=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, username, password FROM users')]"
id=1 username=alice password=alice123
id=2 username=bob password=$2b$12$NPz.bZWk01cw3kRoi6v9Veb/Nlyz8KWEued6Eef2NHGrKG6UVDfK

G:\MIS5303 Securing Software, S2B1\lab 5>python -c "import sqlite3; [print('id=%s owner_id=%s content=%s' % r) for r in
sqlite3.connect('app.db').execute('SELECT id, owner_id, content FROM notes')]"
id=1 owner_id=1 content=Patient reports lower back pain, 6/10.
id=2 owner_id=2 content=gAAAAABqf0ARon2BCz55MR6p6sq3B5H3xv3D6QfAQBSR0kPb3N3Mt0bv90zyHR8IGY8u4IIId9tfB3wSMI3nwxUYyPxiskvRV
o0tTI_10jPT__m1wi_jY0eDhpTFnLj5Kq7xQ6pFKS6VH

G:\MIS5303 Securing Software, S2B1\lab 5>

```

Figure 4: alice's old plaintext note next to bob's new Fernet ciphertext (gAAAAA...) in the same database.

## Secure(?) Notes App

Logged in as bob - [Logout](#)

**Your Notes**

- Patient reports lower back pain, 6/10.

Figure 5: the app decrypts on display, so bob sees his note as normal readable text even though the database only holds ciphertext.

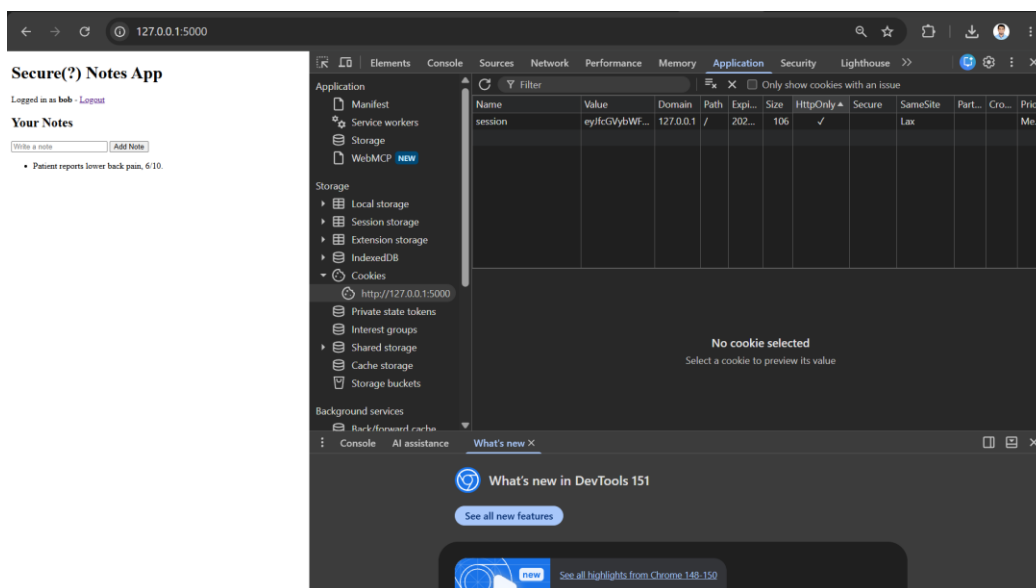


Figure 6: the session cookie now shows HttpOnly enabled and SameSite set to Lax.



### 3. Architect's Perspective — Crypto Design and Justification

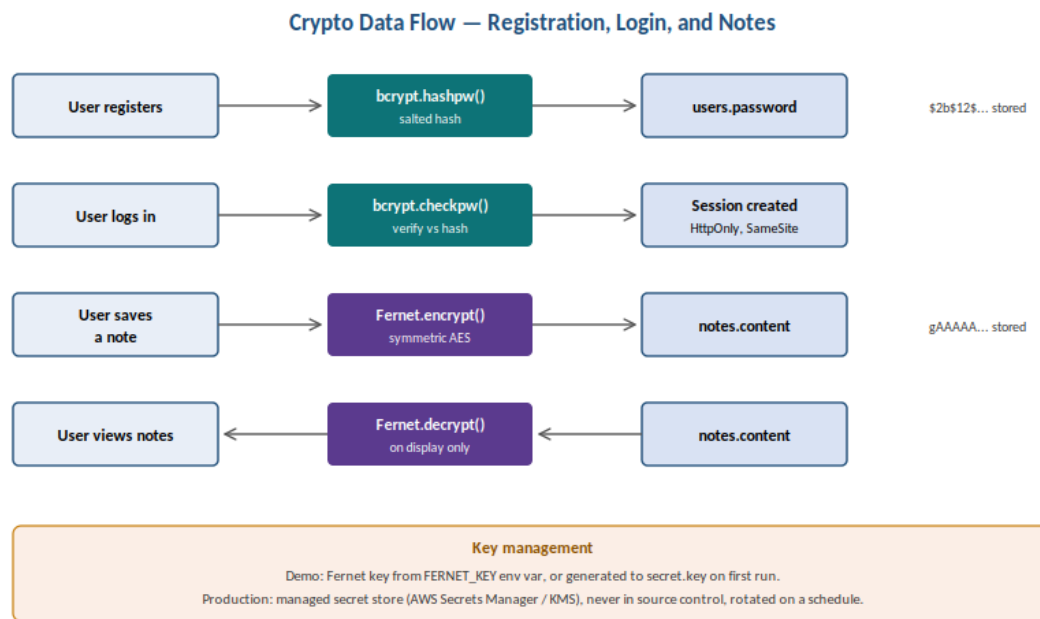


Figure 7: crypto data-flow diagram showing where hashing, encryption, and decryption occur.

| Data / Field | Protection                          | Library               | Justification                                                                                                                    |
|--------------|-------------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Passwords    | Hash + salt (bcrypt, cost 12)       | bcrypt                | Protects against offline attacks if the DB is stolen; salt defeats rainbow tables; adaptive cost keeps pace with faster hardware |
| Notes        | Symmetric encryption (Fernet / AES) | cryptography (Fernet) | Prevents data leakage if the DB or a backup is stolen; authenticated encryption also detects tampering                           |
| Sessions     | Secure cookie flags + timeout       | Flask config          | HttpOnly stops JS/XSS reading the cookie; SameSite limits CSRF; a 15-min idle timeout limits a stolen session's window           |

#### Key management (Part 3 requirement)

What I encrypted, and why: The “note content” field is the field that will most likely contain info typed freely by the client, including personal or clinical information, and as such is the highest value field if the database is ever exported. The reason why the password is hashed and not encrypted is that there is no need to decrypt the password, only to check if it is correct; so the one-way, salted hash is stronger and easier to control.

How I'd use key management in production: In this particular lab, the Fernet key is read from a FERNET\_KEY environment variable - if it were not, a secret.key file would be generated locally (with a console warning - a demo thing only). In production, instead, the key must be generated and stored in a secret store (AWS Secrets Manager, AWS KMS, whatever you prefer), shouldn't be in source control or next to application code, shouldn't be retrieved by the application during runtime with wide open IAM permissions, and should be rotated according to a plan with existing data re-encrypted under the new key. There should be log and alert for access to the key itself, because if the key is compromised, all the notes that it protects are compromised.

#### Why these libraries, specifically

Both bcrypt and argon2 would be a good, acceptable password hasher, and will both work for this lab. So, I opted for bcrypt, since it is mature, has an audited, widespread, simple to reason-about and tune



Python binding (bcrypt package), and is not memory hard, meaning that its cracking attacks are simply slower, not more expensive (not that I'm considering using it in a production system to store MediTrack-like health data, but). Either is certainly better than a fast general hash function (SHA-256 or MD5), which don't have any inherent slowness or salting and are therefore inappropriate for passwords.

Fernet vs alternatives: Fernet (from the cryptography library), because it's a high level API which generates an IV for me, encrypts with HMAC for integrity checking, formats the keys for me - basically very little chance of making low level mistakes like reusing IVs, or forgetting to use integrity protection. The other option would be directly using AES-GCM through the same library, which would be slightly more efficient, but less 'set and forget' due to the responsibility of keeping track of nonces, which is often the root cause of the real world crypto bugs. Using AES-GCM may be better for a higher throughput clinical-notes field; Fernet's simplicity and safety trade-off is a better choice for a smaller field.

### **Threats addressed, and what remains**

This covers protection against credential theft from a stolen database (hashed credentials), protection against bulk data exposure from a stolen database or backup (encrypted credentials), and protection against basic session/cookie theft (Secure Flags). Currently doesn't deal with: Infrastructure changes to enable data to be encrypted in transit (beyond TLS), Infrastructure monitoring of who accesses what note at the field level, or whether the server running the app is fully compromised while a note is being viewed (key in memory). These are reasonable further steps once controls currently in place are in place.

## 4. Individual Reflection

In this lab I created two crypto protections for this app, one for password authentication with bcrypt and the other for the note content with Fernet. The most difficult part of them was the ordering. The note will need to be encrypted before it's added to a database and decrypted when viewing it, not put back into the database in plain text. In addition, I had to consider the fact that alice's old password for the baseline app where she never had even thought of security had also sat in the same database in clear text as bob's new one so a prime example of how complicated things can get if one adds security to something already built, versus building it right away.

To be honest I didn't even bother to trust the code, I just checked the database to make sure I got the fix. When both SELECT queries were executed on both apps, the result was clear — alice's password was still plain text (alice123), but bob's new password was in the form of a bcrypt hash that began with 2b\$12 . In the same manner, the notes: alice's remained readable and bob's became a Fernet cleartext. That was the best indication whether the protection actually worked or if the code was correct - they were both outputted in the same output.

The key management is one thing that I know that is a limitation. In this lab, you will be creating and saving the Fernet key on the local file, and that's not considered safe in production, right? A real system would retrieve the key from a secret store such as AWS Secrets Manager, not be stored next to the code, and use rotation, too.

Many apps still have crypto problems in production because it is easy to want something simple and quick versus taking the time to find the right tool and it's easy to believe that just because the passwords are hashed then you're done, but other sensitive fields such as notes are not protected and that is an issue.

## 5. Peer Review

*Reviewer: Z N Zakaria (Student ID 240134). Peer feedback on this Lab 5 submission.*

### **Are the hashing/encryption changes robust?**

Yes – bcrypt with cost 12 and Fernet are well vetted options, and well demonstrate the transformation from plaintext to hash/ciphertext before and after the process. One suggestion: also display a failed logon with a failure password, to ensure that seeing a bcrypt checkpw fail with an incorrect password and successful login is indeed that (and not just that logins succeeded).

### **Is the design justified and realistic?**

The separation of the keys and architect table is a proper progression especially for a small application, moving the Fernet key to an environment variable now, and a managed secret store down the road. It might be improved by specifying the key-rotation interval (e.g. 90 days) instead of just stating that it should be evaluated or rotated.

### **Polishing / presentation**

Screenshots are clear, and the side-by-side shots (plaintext vs. hash/bob vs. alice), where same output stands side-by-side, are a good way to indicate the change. Make a suggestion for the inclusion of figure numbers on each image.

Reviewer signature: Z N Zakaria    Date: 15-Aug-26